

CS112: Wednesday 23 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Monday we built `Pair`: the header and the implementation, `Pair::` binding them, `public` as a promise and `private` as the freedom to change your mind, and `const` on every getter.

Today: `<<` is a function, and you can write it. `cout << p` does not work yet and there is no reason it should not. The operators are not reserved words with fixed meanings, they are function names with unusual spelling, and this one lands in a place that surprises people: outside the class.

PART

1. Why, before how
2. The broken version first
3. Where the function has to live
4. The Python bridge

Part 1. Why, before how

```
p.print(cout); cout << endl;    // what you can do now
cout << p << endl;             // what you want
```

Both print a pair. Only one of them reads like the rest of your program.

Part 2. The broken version first

Write it returning `void`, deliberately, and run into the wall together:

```
void operator<<(ostream &out, const Pair &p);

cout << p << endl;    // error
```

Talk to your neighbor. Why does adding `<< endl` break it?

Because `cout << p << endl` is `(cout << p) << endl`. The left half has to hand back something that another `<<` can be applied to, and `void` is

not something.

```
ostream& operator<<(ostream &out, const Pair &p) {  
    out << "(" << p.getFirst() << ", " << p.getSecond() << " )";  
    return out;  
}
```

A reference, not a copy, because streams cannot be copied.

Part 3. Where the function has to live

Outside the class. A method belongs to the thing on the left of the operator, and the thing on the left is `cout`. A method of `Pair` would describe `p << something`, which nobody writes.

So the function takes both operands as parameters, in the order they appear: the stream first, the object second, `const` because printing does not change it.

Part 4. The Python bridge

This is `__str__`, spelled with operators instead of with a method name. Python gave you one way to say "here is how this prints"; C++ gives you the same idea by letting you write the operator itself.